

Introduction to Computer Graphics and Visualization Assignment

May 18, 2026

1 Getting started

In this first part of the assignment you will set up the environment for your ray caster implementation. The provided framework is written in C++.

To get started:

- Download the source code of the ray caster framework from Brightspace.
- Look at the included README file for a description of the source files and how to compile and run the ray caster.
- The directory `scenes/` contains the scene files from the visible test cases on Themis with the corresponding correct output images.
The directory `scenes/other/` contains fancier scene files not used in any sub-assignment, but which you can render after completing the assignment to show the full capabilities of your ray caster. You are encouraged to create your own scene files to test your implementation.
- Compile the application and test it using the supplied example scene `other/scene01.json`. The image in Figure 1 should be created.

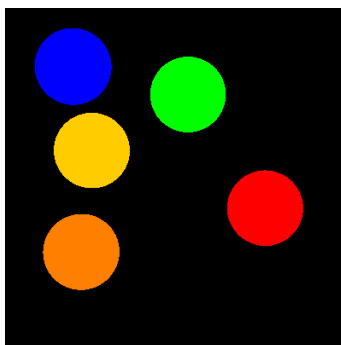


Figure 1: *scene01.json*, as rendered using the unmodified framework.

- The ray tracer currently renders spheres using a fixed radius, and their color is defined in their `Material` data member. We will change this later.
- Look at the source code of the classes to understand the application. Of particular importance is the file `triple.h` which defines mathematical operators on vectors,

points and colors. The actual ray casting algorithm is implemented in `scene.cpp`. The JSON-based scene files are parsed in `raycaster.cpp`.

- While you are free to create your own scene files, do **not** modify the constants in the source code, such as the attribute names used in the JSON parsing. If these constants have changed, Themis may no longer be able to correctly judge your submission.
- The Volume objects in the scene files have a property `volume_data_path` which is a string to a `.dat` file containing meta-information of the volume's data. When specifying a relative path (i.e. a path starting with `..`, this path is relative to your current working directory. The provided scene files can, e.g., be used as follows. First, make `icgv_framework/` your working directory. Run `build.sh` which creates a directory `build/` with an executable `ray` in it. Run, still from the same working directory, `build/ray scenes/05_intersection/1.json`.

Working with Themis

This assignment consists of multiple sub-assignments in which you will be repeatedly adding new functionality to a given framework. After completing a sub-assignment, compress your project into a `zip` or `tar.gz` file and submit it to the corresponding sub-assignment on Themis. Include **only** the `CMakeLists.txt` and `src` directory in your archive.

It may take a minute for Themis to compile and judge your submission, especially if a sub-assignment has many test cases. After a submission, you will have to wait 5 minutes before making another submission for that sub-assignment. You have unlimited attempts at each sub-assignment. The sub-assignments are intended to be completed in order, and submitting a later solution to an earlier sub-assignment may not necessarily work.

Note: In very rare cases, Themis is known to report “Timeout: CPU-time limit exceeded” despite the program finishing execution within the time limit. If this happens, you may ask the teaching assistants to rejudge your submission, or wait 5 minutes and resubmit.

This assignment does not have code quality/documentation points. That said, try to keep your code clean of course.

Issues along the way

The assignment runs until the end of the course. **There is an intermediate deadline for Section 2.** However, note that it is quite long and each new component depends on the previous to work. So, we advise that you get started early, so that in case you run into issues or get stuck along the way, you can come to one the labs in the final few weeks of the course and the TAs present will help you.

2 Ray casting spheres and diffuse illumination (5 points)

In this assignment, your application will produce a first image of a 3D scene using a basic ray casting algorithm. The intersection calculation together with normal calculation will be the groundwork for the illumination.

- Your ray caster only needs to support spheres (see `objects/shapes/sphere.cpp`). Each sphere is given by its center, its radius, and its surface parameters (see `material.h`).
- A point light source is defined by its position (x, y, z) and color (r, g, b) . In the example scene `other/scene01.json` a single white light source is defined.
- The viewpoint is defined by its position (x, y, z) , direction (look at) and field of view.
- The scene description is read from a file in `raycaster.cpp`.

2.1 Sphere intersection and the ambient component (1.0 point)

The function `Sphere::intersect` in `sphere.cpp` contains a placeholder implementation for sphere intersection. Replace it with a real intersection calculation, with the function correctly returning either `nullopt`, or `Hit(t, N, shared_from_this())` with the correct value for `t`. The normal `N` can for now remain at its default value, i.e. $(0, 0, 0)$. Use `Sphere`'s private member function `quadratic` in your implementation.

In the function `Scene::shadeHit`, the returned color is the material color. Replace this by returning the ambient component I_A of the illumination model. This component should not be influenced by any lights in the scene. Figure 2 shows sphere intersection and ambient lighting for two different scenes.

Themis: Once you have completed this part, create an archive with the `CMakeLists.txt` and `src` directory, submit it to Themis and verify that it passes all test cases. If so, you can move on to the next part.

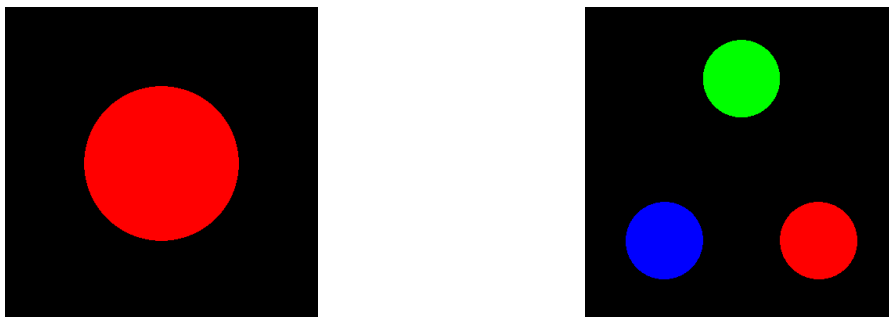


Figure 2: Spheres with different radii. Ambient component only.

2.2 Sphere with the ambient and diffuse components (1.5 points)

In this sub-assignment, we will add the diffuse component to the lighting implementation. As the diffuse component uses the normal at the point of intersection, the normal calculation has to be implemented first.

2.2.1 Normal calculation

Complete the function `Sphere::intersect` in the file `sphere.cpp` by implementing the calculation of the normal $\mathbf{N}$. Implement the illumination calculations in the `Scene::shadeHit` member function. It is necessary that the normal is directed logically towards the ray's origin, i.e., $\mathbf{N} \cdot \mathbf{V} \geq 0$. Ensure that this is the case in your implementation.

As the normal is not yet used by the ray casting implementation, the ray caster should produce identical images as before.

Before adding the diffuse component, it is useful to first test the normal calculation by (temporarily) implementing normal shading. In normal shading, the normals of an object are visualized by mapping them to a color.

Extend `scene.cpp` to map the normal to a color using this equation:

$$\mathbf{C} = \frac{\mathbf{N} + 1}{2},$$

where $\mathbf{C}$ is the color and $\mathbf{N}$ is the normal. Note that this maps the range of a normal-component, i.e., $[-1, 1]$, to the range of a color-component, i.e., $[0, 1]$. Return $\mathbf{C}$ as the result of the `Scene::shadeHit` function, thus (temporarily) ignoring the ambient component.

Rendering `1.json` should produce the image in Figure 3.



Figure 3: Sphere with normal mapping.

If the normals have the correct values, then **remove the normal mapping** and continue.

2.2.2 Adding the diffuse component

For this sub-assignment, you may assume that the scene contains exactly one light source.

The normal, as implemented in Section 2.2.1, can now be used in the calculation of the diffuse component I_D of the illumination model.

Modify the function `Scene::shadeHit` to return the sum of the ambient and diffuse components.

Important: for the model to work, $\mathbf{N} \cdot \mathbf{V} \geq 0$ needs to hold. If this is not the case, then we flip $\mathbf{N}$ (i.e., make sure the normal is always oriented towards the ray's origin). Now, there are two cases:

- The light and camera are both on the outside of the sphere or both on the inside of the sphere, in which case we add the diffuse component.
- The camera is on the inside of the sphere and the light on the outside, or vice versa. In this case the light illuminates the other side of the object (not the side the camera sees) and we only add the ambient component.
- No other cases need to be considered.

These two cases are distinguished by checking $\mathbf{N} \cdot \mathbf{L}$ after the potential normal flip to make sure $\mathbf{N} \cdot \mathbf{V} \geq 0$, where $\mathbf{L}$ is the unit vector from the point of intersection towards the light.

Figure 4 shows an example of using both the ambient and the diffuse components.

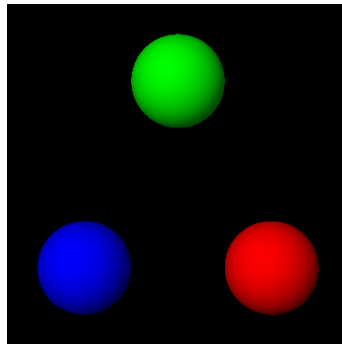


Figure 4: *Spheres with ambient and diffuse shading.*

Themis: Once you have completed this part, create an archive with the `CMakeLists.txt` and `src` directory, submit it to Themis and verify that it passes all test cases. If so, you can move on to the next part.

2.3 Multiple lights (1.0 point)

Extend the `Scene::shadeHit` function such that the ray caster now takes all light sources into account. As in Section 2.1, the ambient component should not be influenced by any lights. Note that the ray caster should also produce the correct output when no light sources are present. Figure 5 shows an example scene with multiple lights.

Themis: Once you have completed this part, create an archive with the `CMakeLists.txt` and `src` directory, submit it to Themis and verify that it passes all test cases. If so, you can move on to the next part.

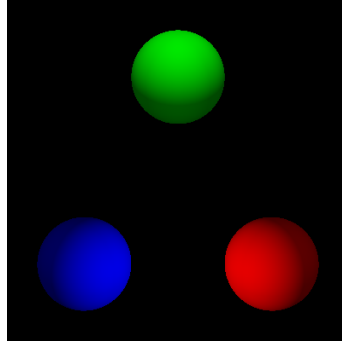


Figure 5: Spheres with diffuse shading and multiple lights.

2.4 Shadows (1.5 points)

The framework is able to read an (optional) parameter from a scene file: `Shadows`. This is a boolean value which it stores in the `Scene` class as `renderShadows`.

Modify the lighting calculation in the `Scene::shadeHit` function to render shadows, depending on whether `renderShadows` is `true` or `false`.

The general approach for detecting shadows is to test whether a ray from the hit point to the light source intersects other objects. Only when this is *not* the case, the light source contributes to the lighting.

Use the provided `Scene::intersectObjects` function to cast a shadow ray.

Hint: Make sure that the intersection of the shadow ray and the other object, if it exists, is not farther from the main object than the light source itself.

To avoid shadow acne, use the method described in the *Ray Tracing* lecture slides. In this implementation, use the `epsilon` value provided in `scene.h`, i.e. 10^{-3} . Make sure to take the direction of the normal into account.

Figure 6 shows a sphere casting a shadow on a larger sphere.

Themis: Once you have completed this part, create an archive with the `CMakeLists.txt` and `src` directory, submit it to Themis and verify that it passes all test cases. If so, you can move on to the next part.

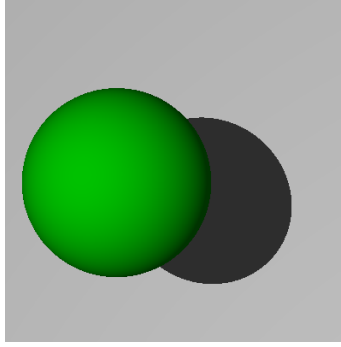


Figure 6: *The shadow of a sphere on a larger sphere.*

3 Volume Raycasting (5 points)

We will now add functionality for volume raycasting, and eventually integrate this with ray tracing from above.

3.1 Intersecting the volume domain (0.5 points)

The first step to volume visualization is rendering the volume's bounding box. To do this, we determine the entry and exit points of a ray intersecting an axis-aligned bounding box (AABB) that represents the volume domain. The AABB is defined by two corner points **aa** and **bb**, as in `volume.h`. The ray is defined by its origin **O** and direction vector **D**.

For each axis, compute the t -values at which the ray intersects the two bounding planes. For the x -axis (and accordingly for y and z):

$$t_x^{aa} = \frac{aa_x - O_x}{D_x}, \quad t_x^{bb} = \frac{bb_x - O_x}{D_x}$$

Note: you do not need to handle the case where a ray direction component is zero explicitly. Floating-point division by zero produces $\pm\infty$, which behaves correctly in comparisons (`std::min` and `std::max` below).

Since the ray direction may be negative, the low t -value does not necessarily correspond to the near intersection. Therefore, take the per-axis near and far values as:

$$t_x^{\text{near}} = \min(t_x^{aa}, t_x^{bb}), \quad t_x^{\text{far}} = \max(t_x^{aa}, t_x^{bb})$$

The entry and exit points of the ray into the AABB are then:

$$t^{\text{near}} = \max(t_x^{\text{near}}, t_y^{\text{near}}, t_z^{\text{near}}), \quad t^{\text{far}} = \min(t_x^{\text{far}}, t_y^{\text{far}}, t_z^{\text{far}})$$

If $t^{\text{near}} > t^{\text{far}}$, the ray misses the AABB and there is no intersection. Additionally, if both t^{near} and t^{far} are negative, the AABB is entirely behind the ray origin. Otherwise, clamp t^{near} to zero to ensure nothing behind the camera is returned.

Implement `Volume::intersect` using the method above.

Now, extend `Scene::trace` by not only calling `intersectObjects`, but `intersectVolumes` as well and treat the volume as an opaque object. For this task, use the framework's placeholder color as returned by `shadeSegment`. You can assume the volumes do not overlap. At this point, you should be able to render a gray-ish box with a uniform color, as in Figure 7.

Themis: Once you have completed this part, create an archive with the `CMakeLists.txt` and `src` directory, submit it to Themis and verify that it passes all test cases. If so, you can move on to the next part.

3.2 Compositing (1.0 point)

Now, we will implement front-to-back compositing for the individual segments we determined. For this, we walk along the ray front-to-back while sampling the scalar density, obtaining color and opacity from the transfer function, and compositing:

$$\begin{aligned} C_i &= C_{i-1} + (1 - O_{i-1})c_i \\ O_i &= O_{i-1} + (1 - O_{i-1})o_i \end{aligned} \tag{1}$$

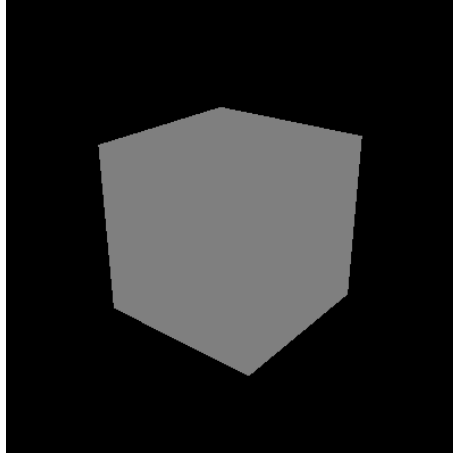


Figure 7: A very gray box.

where C_{i-1} is the already accumulated color and c_i is the color of the current sample. The opacity of a color c_i is denoted here as o_i . Note that in the placeholder implementation, `shadeSegment` returns a pre-multiplied color.

Replace the placeholder implementation in `shadeSegment` by a ray marching loop. Do not start at t_{min} and increment by time step Δt , as that accumulates floating point errors. Instead, determine the number of steps you need to take in the volume (make sure to cover the whole segment!), and compute the new value of t at the start of each iteration. Sample the volume at the point (and pass in `Scene's volumeTrilinear` as the second parameter). For now, `Volume::sample` has a placeholder implementation, but you should still end up with something like Figure 8.

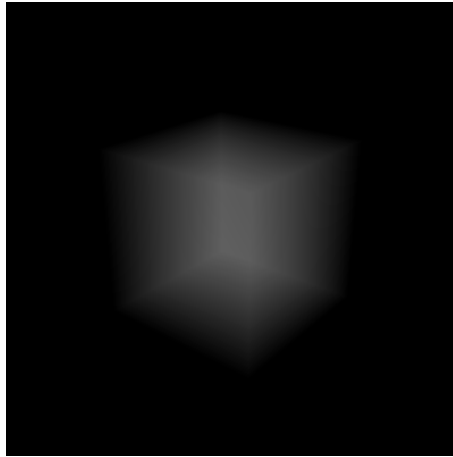


Figure 8: Composited volume with Equation 1.

Just like with spheres, we only implement ambient lighting initially. To do this, multiply the volume's ambient coefficient (stored in its `DensityField`) with the color of the sample.

Optional: Adjust the loop to feature early ray termination (for an opacity O_i greater than 0.99).

Now, the `trace` function needs to be modified to treat volumes as semi-transparent objects instead of opaque ones. Again, use the compositing scheme from Equation 1 to blend the colors from multiple volumes (i.e. `Segments` in code) and an optional (always opaque) sphere.

Optional: Add early ray termination here as well (for an opacity $O_i > 0.99$).

Themis: Once you have completed this part, create an archive with the `CMakeLists.txt` and `src` directory, submit it to Themis and verify that it passes all test cases. If so, you can move on to the next part.

3.3 Sampling the actual data (1.0 point)

Now, it's time to actually see our datasets!

Implement `Volume::sample` by sampling the density at the given point in its `DensityField`. `densityAt` chooses between nearest-neighbor and trilinear interpolation, which you should also complete. In the former, round the percentage coordinate to the nearest grid point, and in both member functions, make sure you handle edge cases correctly!

Once you have the density, call the volume's transfer function as follows:

```
return transferFunction(density);
```

We can call this variable as if it were a function because of its `operator()` member function. Note that the transfer function's colors in the scene file are *not* pre-multiplied colors, but that these are converted to pre-multiplied colors in `TFPoint`'s constructor in `volumes/transferfunction.h`.

And there we have it! If you did everything correctly, you should see what our datasets look like (Figure 9). The provided input scene files already have transfer functions that render the volumes well, but you are very much encouraged to adjust them and figure out how they work. You may be able to improve the visualization, or bring out different details from the datasets.

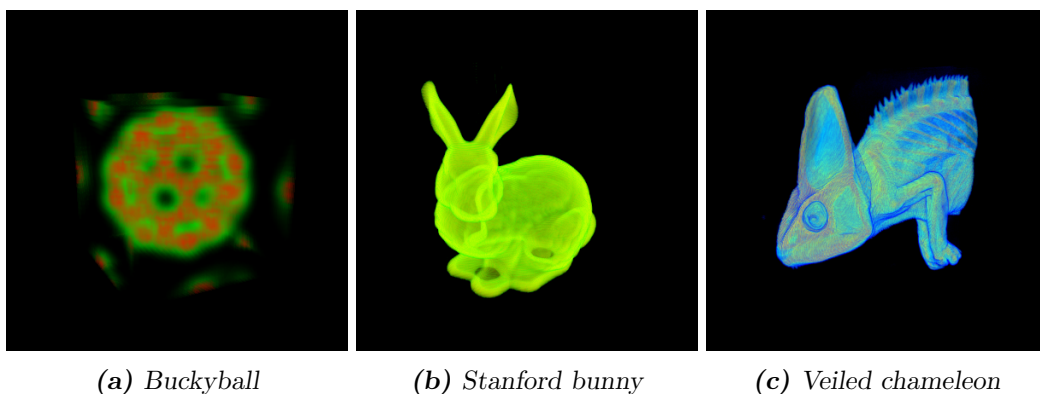


Figure 9: Our three datasets.

Themis: Once you have completed this part, create an archive with the `CMakeLists.txt`

and `src` directory, submit it to Themis and verify that it passes all test cases. If so, you can move on to the next part.

3.4 Diffuse shading (1.0 point)

To add diffuse shading to volumes, we need to be able to approximate its gradient at any point. We can do this by sampling neighboring points to determine the rate of change along each axis. In `Volume::gradient`, compute the gradients ∇f at each point along the ray using the following equation for central differences:

$$\nabla f = \begin{pmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \\ \frac{\partial f}{\partial z} \end{pmatrix} = \begin{pmatrix} \frac{f(x_{i+1,j,k}) - f(x_{i-1,j,k})}{2\Delta x} \\ \frac{f(x_{i,j+1,k}) - f(x_{i,j-1,k})}{2\Delta y} \\ \frac{f(x_{i,j,k+1}) - f(x_{i,j,k-1})}{2\Delta z} \end{pmatrix}, \quad (2)$$

where Δx is the voxel size along the x axis, and f represents the opacity of the volume at a point. Make sure you account for the bounds of the volume correctly, and adjust Δx , Δy and Δz accordingly!

You only need to compute the gradient at points along the ray where opacity is non-zero (larger than `1E-6`).

Note that in areas where the densities are equal (e.g. in empty space) the computed gradients are zero vectors. Depending on the implementation, this can lead to undesirable and inconsistent results. Thus, only take into account gradients with a magnitude above `epsilon` and skip the rest.

We again update `shadeSegment` to add the diffuse component. This is similar to the spheres' diffuse illumination: ensure the normalized gradient vector faces the camera, and add the diffuse component to the accumulating color.

Finally, you should see our volumes being lit from the side as in Figure 10.

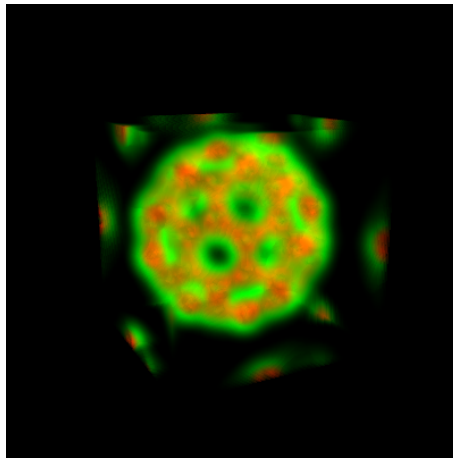


Figure 10: The buckyball with volumetric illumination.

Themis: Once you have completed this part, create an archive with the `CMakeLists.txt` and `src` directory, submit it to Themis and verify that it passes all test cases. If so, you can move on to the next part.

3.5 Shadows (1.0 point)

Before we integrate our (already very capable) volume renderer with the sphere renderer, we will need to add one more feature to match the ray tracer: shadows.

For spheres, we determined if a hit point was illuminated by the lights in the scene or blocked by another sphere. We will do the same for the volume sample points, except that now, it is not as black and white: volumes can partially obscure the light falling on another volume.

We could call `Scene::trace` recursively and use the returned opacity, but that would result in recursive calls, wasting compute time. To speed things up, we'll make use of a separate function, `Scene::traceShadowOcclusion`.

Of course, if the shadow ray hits a sphere, it is fully occluded. If not, call `Scene::shadeSegmentOpacity` on every segment towards the light. Take care to only take into account hits and segments between the point and the light. Here, we can conveniently also employ early ray termination.

In turn, `Scene::shadeSegmentOpacity` is a much-simplified version of `Scene::shadeSegment`. We use the same iteration, and just accumulate the opacity, and not the color. Applying early ray termination here is, again, optional.

Like in Figure 11, you should see spheres casting shadows not only on other objects, but on volumes as well.

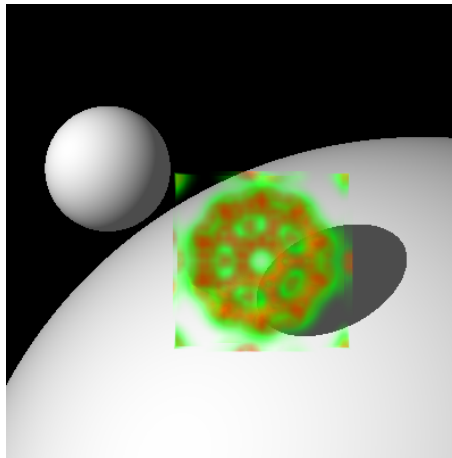


Figure 11: A volume being shadowed by a sphere.

Themis: Once you have completed this part, create an archive with the `CMakeLists.txt` and `src` directory, submit it to Themis and verify that it passes all test cases. If so, you can move on to the next part.

3.6 Sphere and volume integration (0.5 points)

Finally, we will fully integrate our ray caster with the ray tracer from Task 2.

Change the shadow rendering code in `Scene::shadeHit` to match `shadeSegment`, to ensure that volumes can also cast shadows on spheres. You should now be able to render

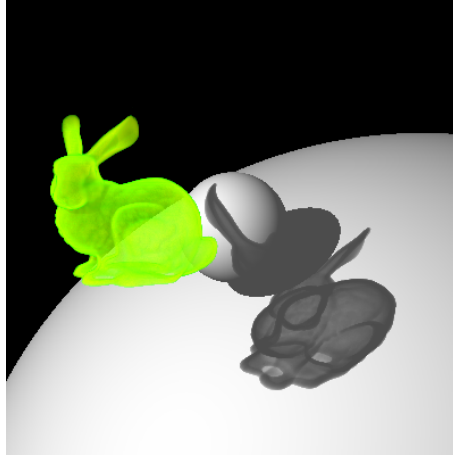


Figure 12: *Two spheres being shadowed by a volume.*

scenes like that in Figure 12.

Themis: Once you have completed this final part, create an archive with the `CMakeLists.txt` and `src` directory, submit it to Themis and verify that it passes all test cases. If so, you've successfully completed the assignment. Congratulations!